

Материалы кафедры ММП факультета ВМК МГУ. Практикум на ЭВМ.

Занятие 02. Функции, модули, классы. Итераторы и генераторы

Занятие провел: Енягин Станислав

Материалы составили: Енягин Станислав,
Акопова Елена, Ищенко Роман
(ischenkorv@my.msu.ru)

Москва, Осенний семестр 2026

На этом занятии мы разберём:

1. **Функции** – аргументы, значения по умолчанию, `*args` / `**kwargs`, области видимости и замыкания;
2. **Функции как объекты** – `lambda`, `map`, `filter`, `reduce` и передача функций другим функциям;
3. **Итераторы и генераторы** – как работает `for`, что значит ленивое вычисление и зачем нужен `yield`;
4. **Модули и классы** – пространства имён, атрибуты, специальные методы и `property`.

```
In [1]: import sys
import math
import builtins
from functools import reduce
from itertools import count
from itertools import cycle
from itertools import accumulate
from itertools import chain
from itertools import groupby
from itertools import product, permutations, combinations
import itertools
```

```
import itertools as it
from itertools import product as prd
from importlib import reload
```

Зачем нужны функции, модули, классы и ленивые вычисления

Функции, модули, классы позволяют организовать и структурировать код, избежать повторяющихся фрагментов.

Кроме того:

- упростить отладку
- упростить создание тестов
- улучшить «понимаемость» кода

Функции: создание, аргументы и области видимости

Обход последовательностей: range, enumerate, zip

```
In [2]: r = range(2, 10, 3)
print(type(r))

for e in r:
    print(e, end=' ')
```

```
<class 'range'>
2 5 8
```

```
In [3]: for index, element in enumerate(list('abcdef')):
        print(index, element, end=' ')
```

```
0 a 1 b 2 c 3 d 4 e 5 f
```

```
In [4]: for a, b in zip([1, 2, 3], 'abc'):
        print(a, b, end=' ')
```

```
1 a 2 b 3 c
```

```
In [5]: for e in zip('abcdef', 'abc'):
        print(e)
```

```
('a', 'a')
('b', 'b')
('c', 'c')
```

```
In [6]: for a, b, c, d in zip('abc', [1,2,3], [True, False, None], 'xyza'):
        print(a, b, c, d)
```

```
a 1 True x
b 2 False y
c 3 None z
```

Создание собственных функций и функции как объекты

```
In [7]: def function(arg_1, arg_2=None):
        """Factorial function"""
        function.a = 4
        print(arg_1, arg_2)

        function(10)
        function(10, 20)
```

```
10 None
10 20
```

Функция – это тоже объект, её имя – просто символическая ссылка:

```
In [8]: f = function
        f(10)

        print(function is f)
```

```
10 None
True
```

Атрибуты функций: встроенные и пользовательские

Функция в Python – обычный объект. Поэтому у неё есть встроенные атрибуты (например, `__doc__`, `__name__`, `__dict__`), а также ей можно добавлять собственные пользовательские атрибуты.

`function.__dict__` хранит именно пользовательские атрибуты, добавленные объекту функции.

```
In [9]: print(dir(f)[: 4], len(dir(f)))
        f.__doc__
```

```
['__annotations__', '__call__', '__class__', '__closure__'] 36
```

```
Out[9]: 'Factorial function'
```

```
In [10]: print(f.__dict__)

        # Создаём пользовательский атрибут у объекта-функции
        f.my_attr = 0
        print(f.__dict__)

        # Атрибут можно удалить так же, как у обычного объекта
        del f.my_attr
        print(f.__dict__)
```

```
{'a': 4}
{'a': 4, 'my_attr': 0}
{'a': 4}
```

Зачем могут пригодиться пользовательские атрибуты функции

Например, функция может хранить дополнительное состояние или метаданные. Ниже `counter.count` хранит число вызовов функции `counter`.

```
In [11]: def counter():
          if not hasattr(counter, 'count'):
              counter.count = 0
          counter.count += 1
          return counter.count

print(counter())
print(counter())
print(counter())
print(counter.count)
```

```
1
2
3
3
```

Рекурсия и глубина вызовов

```
In [12]: def factorial(n):
          return n * factorial(n - 1) if n > 1 else 1

print(factorial(1))
print(factorial(2))
print(factorial(4))
```

```
1
2
24
```

При достижении глубины вызовов, заданной в системе, программа выбросит исключение `RecursionError`

Заданную глубину рекурсии можно узнать и изменить, если есть такая необходимость.

```
In [13]: import sys
          print(sys.getrecursionlimit())

          sys.setrecursionlimit(2000)
          print(sys.getrecursionlimit())
```

```
3000
2000
```

Передача аргументов: изменяемые объекты и значения по умолчанию

При вызове функции параметры связываются с переданными объектами.

Если объект изменяемый (`list` , `dict` , `set`), функция может изменить сам объект, и это изменение будет видно снаружи. Переприсваивание локального имени при этом не меняет внешнюю переменную.

```
In [14]: def function(scalar, lst):
          scalar += 10
          print(f'Scalar in function: {scalar}')

          lst.append(12)
          print(f'List in function: {lst}')
```

```
In [15]: s, l = 5, []
          function(s, l)

          print(s, l)
```

```
Scalar in function: 15
List in function: [12]
5 [12]
```

Опасный случай: изменяемый объект как значение по умолчанию

Значения аргументов по умолчанию вычисляются **один раз – при создании функции**. Поэтому `set()` , `[]` или `{}` в сигнатуре будут одним и тем же объектом между несколькими вызовами функции. Это почти всегда неожиданное поведение.

```
In [16]: def func_1(list_of_items, my_set=set()):
          final_list = list()
          for item in list_of_items:
              if item not in my_set:
                  my_set.add(item)
                  final_list.append(item)
          return final_list

          my_list = [1, 2, 3]
          print(func_1(my_list))
          print(func_1(my_list))
```

```
[1, 2, 3]
[]
```

Правильный шаблон: None + создание объекта внутри функции

Если для каждого вызова нужен новый изменяемый объект, используем `None` как

значение по умолчанию и создаём `set()` / `list()` / `dict()` уже внутри функции.

```
In [17]: def func_1(list_of_items, my_set=None):
        if my_set is None:
            my_set = set()

        final_list = list()
        for item in list_of_items:
            if item not in my_set:
                my_set.add(item)
                final_list.append(item)
        return final_list
my_list = [1, 2, 3]
print(func_1(my_list))
print(func_1(my_list))
```

```
[1, 2, 3]
```

```
[1, 2, 3]
```

Позиционные и именованные аргументы: *args* и **kwargs*

**args* используется для сбора произвольного числа позиционных аргументов.

```
In [18]: def f(a, *args):
        print(type(args))
        print([v for v in [a] + list(args)])

        f(10, 2, 6, 8)
```

```
<class 'tuple'>
```

```
[10, 2, 6, 8]
```

```
In [19]: def f(*args, a):
        print([v for v in [a] + list(args)])
        print()

        f(2, 6, 8, a=10)
        f(2, 3, 4, 5)
```

```
[10, 2, 6, 8]
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[19], line 6
      3     print()
      5     f(2, 6, 8, a=10)
----> 6     f(2, 3, 4, 5)

TypeError: f() missing 1 required keyword-only argument: 'a'
```

***kwargs* используется для сбора произвольного числа именованных (ключевых) аргументов в виде словаря.

```
In [20]: def f(a, *args, **kw):
```

```

    print(type(kw))
    print([v for v in [a] + list(args) + [(k, v) for k, v in kw.items()]])

    f(2, *(6, 8), **{'arg1': 1, 'arg2': 2})

```

```

<class 'dict'>
[2, 6, 8, ('arg1', 1), ('arg2', 2)]

```

```

In [21]: def test(a, *args, b, c=100, **kw):
          print([v for v in [a] + list(args) + [b] + [c] + [(k, v) for k, v in kw.items()]])

          test(1, 2, 3, b=10, d=1000)

[1, 2, 3, 10, 100, ('d', 1000)]

```

Как ограничить способ передачи параметров: / и *

```

In [22]: import math
          #help(math.exp) -> ... exp(x, /) ...

```

```

In [23]: print(math.exp(5))
          #print(math.exp(x=5)) -> TypeError: math.exp() takes no keyword arguments

148.4131591025766

```

Синтаксис заголовка функции:

```

In [24]: def func(positional_only_parameters, /, positional_or_keyword_parameters, *,

```

```

In [25]: def f(a, b, /, c, *, d): pass
          #f(1, 2, 3, 4) -> TypeError: func() takes 3 positional arguments but 4 were
          #f(a=1, b=2, c=3, d=4) -> TypeError: func() got some positional-only argument
          f(1, 2, 4, d=3) # OK
          f(1, 2, d=4, c=3) # OK

```

Области видимости переменных: Local, Enclosing, Global, Builtins

Как Python ищет имя: правило LEGB

```

In [26]: a = []

          def outer_func(x):
              def inner_func(x):
                  a.append(x)
                  return len(x)
              return inner_func(x)

```

```

In [27]: print(outer_func([1, 2]))

```

2

Кто определил имя `len` ?

- на уровне вложенной функции такого имени нет, смотрим выше
- на уровне объемлющей функции такого имени нет, смотрим выше
- на уровне модуля такого имени нет, смотрим выше
- на уровне builtins такое имя есть, используем его

На builtins можно посмотреть

```
In [28]: import builtins

counter = 0
lst = []
for name in dir(builtins):
    if name[0].islower():
        lst.append(name)
        counter += 1

    if counter == 5:
        break

lst
```

```
Out[28]: ['abs', 'all', 'any', 'ascii', 'bin']
```

Кстати, то же самое можно сделать более pythonic кодом:

```
In [29]: list(filter(lambda x: x[0].islower(), dir(builtins)))[: 5]
```

```
Out[29]: ['abs', 'all', 'any', 'ascii', 'bin']
```

Чтение и изменение локальных и глобальных переменных

```
In [30]: x = 2
def func():
    print('Inside: ', x) # read

func()
print('Outside: ', x)
```

```
Inside:  2
Outside: 2
```

```
In [31]: x = 2

def func():
    x += 1 # write
    print('Inside: ', x)

func()
print('Outside: ', x)
```

```

-----
UnboundLocalError                                Traceback (most recent call last)
Cell In[31], line 7
      4     x += 1 # write
      5     print('Inside: ', x)
----> 7 func()
      8 print('Outside: ', x)

Cell In[31], line 4, in func()
      3 def func():
----> 4     x += 1 # write
      5     print('Inside: ', x)

UnboundLocalError: local variable 'x' referenced before assignment

```

```

In [32]: x = 2
def func():
    x = 3
    x += 1
    print('Inside: ', x)

func()
print('Outside: ', x)

```

```

Inside:  4
Outside: 2

```

Изменение глобального имени: ключевое слово `global`

```

In [33]: x = 2
def func():
    global x
    x += 1 # write
    print('Inside: ', x)

func()
print('Outside: ', x)

```

```

Inside:  3
Outside: 3

```

```

In [34]: x = 2
def func(x):
    x += 1
    print('Inside: ', x)
    return x

x = func(x)
print('Outside: ', x)

```

```

Inside:  3
Outside: 3

```

Изменение переменной внешней функции: ключевое слово `nonlocal`

`nonlocal` используется для работы с переменными из объемлющей (внешней) функции, но не глобальной области видимости. Оно позволяет изменять переменные из замыканий.

```
In [35]: a = 0
def out_func():
    b = 10
    def mid_func():
        c = 20
        def in_func():
            global a
            a += 100

            nonlocal c
            c += 100

            nonlocal b
            b += 100

            print(a, b, c)

        in_func()
    mid_func()

out_func()
```

100 110 120

Главный вывод: не надо злоупотреблять побочными эффектами при работе с переменными верхних уровней

Замыкания: как функция запоминает внешнее окружение

```
In [36]: def function_creator(n):
def function(x):
    return x ** n

    return function

f5 = function_creator(5)
f5(2)
```

Out[36]: 32

Объект-функция, на который ссылается `f`, хранит в себе значение `n`

Lambda: короткие анонимные функции

```
In [37]: def func(x): return x ** 2
func(6)
```

Out[37]: 36

```
In [38]: lambda_func = lambda x: x ** 2 # should be an expression
lambda_func(6)
```

Out[38]: 36

```
In [39]: def func(x): print(x)
func(6)
```

6

```
In [40]: lambda_func = lambda x: print(x ** 2) # as print is function in Python 3.*
lambda_func(6)
```

36

sorted и параметр key: функция как аргумент

```
In [41]: lst = [5, 2, 7, -9, -1]

print(sorted(lst))
```

[-9, -1, 2, 5, 7]

```
In [42]: def abs_comparator(x):
return abs(x)

print(sorted(lst, key=abs_comparator))
```

[-1, 2, 5, 7, -9]

```
In [43]: print(sorted(lst, key=lambda x: abs(x)))
print(sorted(lst, key=abs))
```

[-1, 2, 5, 7, -9]

[-1, 2, 5, 7, -9]

Более сложный пример

```
In [44]: lst = [(1, 0), (1, 1), (1, 2), (0, 3), (2, 1)]
print(sorted(lst))
```

[(0, 3), (1, 0), (1, 1), (1, 2), (2, 1)]

```
In [45]: print(sorted(lst, key=lambda x: x[1])) # сортировка по второму элементу
print(sorted(lst, key=lambda x: x[0] * x[1], reverse=True)) # сортировка по
```

[(1, 0), (1, 1), (2, 1), (1, 2), (0, 3)]

[(1, 2), (2, 1), (1, 1), (1, 0), (0, 3)]

filter: отбор элементов по условию

```
In [46]: lst = [5, 2, 7, -9, -1]
```

```
In [47]: f = filter(lambda x: x < 0, lst)
         type(f)
```

```
Out[47]: filter
```

```
In [48]: list(f)
```

```
Out[48]: [-9, -1]
```

map: преобразование каждого элемента

```
In [49]: lst = [(1, 0), (1, 1), (1, 2), (0, 3), (2, 1)]
```

```
In [50]: m = map(lambda x: x[0] * x[1], lst)
         type(m)
```

```
Out[50]: map
```

```
In [51]: list(m)
```

```
Out[51]: [0, 1, 2, 0, 2]
```

Императивный и функциональный стили: скалярное произведение

```
In [52]: def dot_product_imp(v, w):
         result = 0
         for i in range(len(v)):
             result += v[i] * w[i]
         return result
```

```
In [53]: dot_product_func = lambda v, w: sum(map(lambda x: x[0] * x[1], zip(v, w)))
```

```
In [54]: print(dot_product_imp([1, 2, 3], [4, 5, 6]))
         print(dot_product_func([1, 2, 3], [4, 5, 6]))
```

```
32
```

```
32
```

reduce: свёртка последовательности в одно значение

```
In [55]: from functools import reduce

         lst = list(range(1, 10))

         reduce(lambda x, y: x * y, lst)
```

```
Out[55]: 362880
```

Итераторы: как Python обходит данные по одному

элементу

Интуитивно:

- **Iterable (итерируемый объект)** – объект, по которому можно пройти: `list`, `tuple`, `str`, `set`, `range` и т.д.
- **Iterator (итератор)** – отдельный объект, который помнит, где мы остановились.
- `iter(obj)` получает итератор из итерируемого объекта.
- `next(iterator)` просит у итератора следующий элемент.
- Когда элементы закончились, возникает `StopIteration`.

Итератор не обязан заранее хранить все результаты. Он может получать следующий элемент только тогда, когда тот понадобится – это и называется **ленивым вычислением**.

Формально итератор реализует протокол `__iter__() + __next__()`, но удобнее сначала помнить простую модель: **итератор = объект, который умеет получать следующий элемент и хранить текущую позицию**.

```
In [56]: r = iter([1, 2, 3])

print(next(r), next(r), next(r))
#next(r) -> StopIteration

print(type(r))

1 2 3
<class 'list_iterator'>
```

Откуда берутся итераторы

Один из способов – вызвать `iter(...)` у итерируемого объекта. Некоторые встроенные функции (`enumerate`, `zip`, `map`, `filter` и др.) сразу возвращают итераторы.

```
In [57]: r = enumerate([1, 2, 3])
print(type(r))

print(next(r), next(r), next(r))
#next(r) -> StopIteration

<class 'enumerate'>
(0, 1) (1, 2) (2, 3)
```

Как цикл for использует iter и next

`for` не обязан работать с индексами. Он получает итератор через `iter(...)` и затем вызывает `next(...)`, пока не встретит `StopIteration`.

Поэтому `for` работает даже с объектами без индексов, например с `set`.

```
In [58]: for e in {1, 2, 3}:  
         print(e, end=' ')
```

1 2 3

Как он работает на самом деле:

```
In [59]: iterator = iter({1, 2, 3})  
while True:  
    try:  
        i = next(iterator)  
        print(i, end=' ')  
    except StopIteration:  
        break
```

1 2 3

Встроенные источники итераторов

Примеры объектов/функций, которые дают итераторы:

- `enumerate`
- `zip`
- файловые объекты, полученные через `open`
- `reversed`
- `map`
- `filter`

У настоящего итератора `iter(iterator)` возвращает его же самого. Это удобно для проверки: **iterator является iterable, но не каждый iterable является iterator.**

```
In [60]: i = iter([1, 2, 3])  
  
print(next(i), next(iter(i)), next(i))
```

1 2 3

Генераторы: удобный способ создавать ленивые итераторы

Генератор – это специальный вид итератора.

Интуитивно генератор можно представить как функцию, которая умеет **останавливаться, запоминать своё состояние и продолжать работу с того же места.**

Генераторы создаются двумя основными способами:

1. генераторным выражением: `(x ** 2 for x in range(5))` ;
2. функцией с `yield` .

Как и другие итераторы, генератор выдаёт элементы по одному и не обязан заранее создавать всю последовательность в памяти. Поэтому генераторы особенно полезны для больших данных и потоковой обработки.

Почему генератор экономит память

Список создаёт все элементы сразу, а генератор хранит в основном состоянии вычисления. Поэтому размер самого объекта-генератора почти не зависит от числа будущих элементов.

`sys.getsizeof` здесь показывает размер самих контейнеров/объектов, а не полный рекурсивный объём всех связанных объектов.

```
In [61]: import sys

my_list = [num for num in range(1, 1000000)]
my_generator = (num for num in range(1, 1000000))

print('list:', sys.getsizeof(my_list), 'bytes')
print('generator:', sys.getsizeof(my_generator), 'bytes')
```

```
list: 8448728 bytes
generator: 112 bytes
```

```
In [62]: gen = (x**2 for x in range(5))

print(next(gen), next(gen))
print(type(gen))
```

```
0 1
<class 'generator'>
```

yield: отдать значение и поставить функцию на паузу

`yield` похож на `return` тем, что отдаёт значение вызывающему коду, но есть важная разница: после `yield` выполнение функции **не заканчивается**, а приостанавливается.

При первом `next(generator)` функция выполняется до первого `yield` . Локальные переменные и текущая позиция сохраняются. Следующий `next(...)` продолжает выполнение сразу после предыдущего `yield` .

Когда функция-генератор завершается, дальнейший `next(...)` приводит к `StopIteration` .

Пример функции-генератора

```
In [63]: def my_range(n):
        yield 'You really want to run this generator?'

        i = -1
        while i < n:
            i += 1
            yield i
```

```
In [64]: gen = my_range(3)
        while True:
            try:
                print(next(gen), end='  ')
            except StopIteration: # we want to catch this type of exceptions
                break
```

You really want to run this generator? 0 1 2 3

```
In [65]: for e in my_range(3):
        print(e, end='  ')
```

You really want to run this generator? 0 1 2 3

Генераторы и метод send: передача значения внутрь

Выражение `y = yield x` работает в двух направлениях:

1. `yield x` отдаёт `x` наружу и ставит генератор на паузу;
2. когда генератор возобновляют, переданное внутрь значение становится результатом выражения `yield` и попадает в `y`.

`next(gen)` эквивалентен `gen.send(None)`. После первого запуска можно использовать `gen.send(value)`, чтобы передать произвольное значение в точку, где генератор остановился на `yield`.

```
In [66]: def create_gen():
        for x in range(5):
            y = yield x
            print(f'y = {y}', end='  ')
```

```
In [67]: gen = create_gen()

        print(next(gen), end=' | ')
        print(next(gen), end=' | ')

        print(gen.send(10), end=' | ')
        print(next(gen), end=' | ')
```

0 | y = None 1 | y = 10 2 | y = None 3 |

range: iterable, но не iterator

`range(...)` можно использовать в `for`, поэтому это **итерируемый объект**. Но

сам `range` не хранит текущую позицию обхода и у него нет `__next__`.

`iter(range(...))` создаёт отдельный итератор. Каждый новый вызов `iter(range(...))` может дать новый независимый обход с начала.

```
In [68]: print('__next__' in dir(zip([], [])))  
print('__next__' in dir(range(3)))
```

True
False

Особенности объектов `range`:

- значения не создаются все сразу: `range` хранит границы и шаг;
- объект `range` неизменяем;
- поддерживает `len`, индексацию и срезы;
- по одному и тому же `range` можно итерироваться многократно, потому что для каждого обхода создаётся новый итератор.

itertools: готовые инструменты для работы с итераторами

Модуль itertools: примеры

```
In [69]: from itertools import count  
  
for i in count(start=0):  
    print(i, end=' ')  
    if i == 5:  
        break
```

0 1 2 3 4 5

```
In [70]: from itertools import cycle  
  
count = 0  
for item in cycle('XYZ'):  
    if count > 4:  
        break  
    print(item, end=' ')  
    count += 1
```

X Y Z X Y

```
In [71]: from itertools import accumulate  
for i in accumulate(range(1, 6), lambda x, y: x * y):  
    print(i, end=' ')
```

1 2 6 24 120

```
In [72]: from itertools import chain  
  
for i in chain([1, 2], [3], [4], '56', "annn"):  
    print(i, end=' ')
```

1 2 3 4 5 6 a n n n

```
In [73]: from itertools import groupby

vehicles = [('Ford', 'Taurus'), ('Dodge', 'Durango'),
            ('Chevrolet', 'Cobalt'), ('Ford', 'F150'),
            ('Dodge', 'Charger'), ('Ford', 'GT')]

sorted_vehicles = sorted(vehicles)

for key, group in groupby(sorted_vehicles, lambda x: x[0]):
    for maker, model in group:
        print('{model} is made by {maker}'.format(model=model, maker=maker))

    print ("**** END OF THE GROUP ***\n")
```

Cobalt is made by Chevrolet
**** END OF THE GROUP ***

Charger is made by Dodge
Durango is made by Dodge
**** END OF THE GROUP ***

F150 is made by Ford
GT is made by Ford
Taurus is made by Ford
**** END OF THE GROUP ***

```
In [74]: from itertools import product, permutations, combinations
print(list(product("AB", repeat=2)))
print(len(list(product("AB", repeat=10))))
print(list(permutations("ABC")))
print(list(combinations("ABC", 2)))

[('A', 'A'), ('A', 'B'), ('B', 'A'), ('B', 'B')]
1024
[('A', 'B', 'C'), ('A', 'C', 'B'), ('B', 'A', 'C'), ('B', 'C', 'A'), ('C',
'A', 'B'), ('C', 'B', 'A')]
[('A', 'B'), ('A', 'C'), ('B', 'C')]
```

Модули: импорт и пространства имён

Любой файл с расширением `.py` является модулем

Оператор `import` импортирует модуль и создаёт ссылку на модуль в текущей области видимости:

- ищет в `sys.path` имя
- компилирует в байт-код (если версия уже неактуальна)
- выполняет код (только первый раз)

```
In [75]: import itertools
```

```
import itertools as it

from itertools import product as prd

# from itertools import *
```

Так как модули тоже объекты, у них тоже есть атрибуты

```
In [76]: import itertools
print(itertools.__name__)

# print(my_module.__path__)
```

itertools

У модулей, переданных на выполнение интерпретатору, `__name__ == '__main__'`

```
In [77]: if __name__ == '__main__':
        # Код, выполняющийся только если именно этот модуль передан интерпретатору
        pass
```

```
In [78]: # Если модуль нужно перезагрузить принудительно
from importlib import reload
reload(itertools)
```

Out[78]: <module 'itertools' (built-in)>

Классы: объекты, атрибуты и специальные методы

Класс объявляется с помощью ключевого слова `class`

```
In [79]: class MyClass:
        def __init__(self, value):
            self.value = value

        def get_value(self):
            return self.value

my_obj = MyClass(2)
print(my_obj.get_value())
```

2

Каждый объект экземпляра наследует атрибуты класса и приобретает свое собственное пространство имен

Создание и инициализация объекта: **new** и **init**

- Главное: `__init__` – не конструктор! Она ничего не создаёт и не возвращает

- Созданием объекта занимается функция `__new__`, переопределять которую без необходимости не надо
- `__init__` получает на вход готовый объект и инициализирует его атрибуты

В отличие от C++, атрибуты можно добавлять/удалять на ходу:

```
In [80]: class Cls:
          pass

cls = Cls()
cls.field = 'field'
print(cls.field)

del cls.field
print(cls.field) # AttributeError: 'Cls' object has no attribute 'field'
```

field

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[80], line 9
      6 print(cls.field)
      8 del cls.field
----> 9 print(cls.field) # AttributeError: 'Cls' object has no attribute 'field'
AttributeError: 'Cls' object has no attribute 'field'
```

Параметр self

- Метод класса отличается от обычной функции только наличием объекта `self` в качестве первого аргумента
- Это то же самое, что происходит в C++/Java (там аналогом `self` является указатель/ссылка `this`)
- Название `self` является общим соглашением, но можно использовать и другое (не надо!)
- Метод класса, не получающий на вход `self` является *статическим*
- Статические методы часто используются для специализированного создания объектов класса

Доступ к атрибутам: getattr, setattr, delattr

- Для работы с атрибутами есть функции `getattr`, `setattr` и `delattr`
- Их основное преимущество – оперирование именами атрибутов в виде строк

```
In [81]: cls = Cls()

setattr(cls, 'some_attr', 'some')

print(getattr(cls, 'some_attr'))

delattr(cls, 'some_attr')

print(getattr(cls, 'some_attr'))
```

some

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[81], line 9
      5 print(getattr(cls, 'some_attr'))
      7 delattr(cls, 'some_attr')
----> 9 print(getattr(cls, 'some_attr'))

AttributeError: 'Cls' object has no attribute 'some_attr'
```

Магические методы: как объект встраивается в синтаксис Python

- Магические методы придают объекту класса определённые свойства
- Такие методы получают `self` вызываются интерпретатором неявно
- Например, операторы – это магические методы

Рассмотрим несколько примеров:

```
In [82]: class Cls:
    def __init__(self): # initialize object
        self.name = 'Some class'

    def __repr__(self): # str for printing object
        return 'Class: {}'.format(self.name)

    def __call__(self, counter): # call == operator() in C++
        return self.name * counter

cls = Cls()
print(cls)
print(cls(2))
```

```
Class: Some class
Some classSome class
```

Class magic methods

Ещё примеры магических методов:

```
In [83]: class Cls():
```

```

def __add__(self, other): pass # x + other

def __sub__(self, other): pass # x - other

def __mul__(self, value): pass # x * value

def __rmul__(self, value): pass # value * x

def __int__(self): pass # int(x)

def __getitem__(self, index): pass # x[index]

def __setitem__(self, index, value): pass # x[index] = value

```

Class magic methods

Как выбирается метод при наличии парного, например, `__add__` и `__radd__` для `x + y`:

1. ищется `x.__add__()`
2. если нашёлся этот метод, то он и вызывается
3. если выполнен успешно, то всё
4. если `x.__add__()` не нашёлся или вернул специальный объект `NotImplemented` («не знаю я, что с вашим `y` делать!»), ищется метод `y.__radd__()`
5. и, наконец, если последний нашёлся – выполняется, если нет – `TypeError`

```

In [84]: class ClsLeft:
        def __add__(self, add):
            print(f'__add__ y {self.__class__.__name__}')
            return NotImplemented

        class ClsRight:
            def __radd__(self, add):
                print(f'__radd__ y {self.__class__.__name__}')
                return ClsRight()

ClsLeft() + ClsRight()
print()

```

```

__add__ y ClsLeft
__radd__ y ClsRight

```

Свойства (property): управляемый доступ к атрибутам

Протокол свойств позволяет направлять операции чтения, записи и удаления для отдельных атрибутов отдельным функциям и методам

```
attribute = property(fget, fset, fdel, doc)
```

```
In [85]: class Cls():
    def __init__(self):
        self._value = 0

    def getValue(self):
        print(f'Getting value')
        return self._value

    def setValue(self, val):
        print(f'Setting value to {val}')
        self._value = val

    value = property(getValue, setValue, None, 'VALUE')

cls = Cls()

print(cls.value)

cls.value = 10
print(cls.value)
```

```
Getting value
0
Setting value to 10
Getting value
10
```

Как сделать экземпляр класса итератором

Класс, который должен поддерживать магические методы `__iter__` и `__next__`

```
In [86]: class iter_example:

    def __init__(self, cnt):
        self.max = cnt

    def __iter__(self):
        self.curr = 0
        return self

    def __next__(self):
        if self.curr < self.max:
            self.curr += 1
            return self.curr ** 2

        raise StopIteration

a = iter(iter_example(5))
print(*list(a))
```

```
1 4 9 16 25
```

Спасибо за внимание!